

Plataforma SaaS de gestión para bares, restaurantes y eventos

Todo lo acordado hasta hoy — visión, arquitectura, modelo de datos, seguridad, modo offline, fiscalidad dominicana y diseño de interfaces — consolidado para que el equipo lo revise y lo discuta *antes* de escribir la primera línea de código.

Para revisión del equipo

Versión 0.1

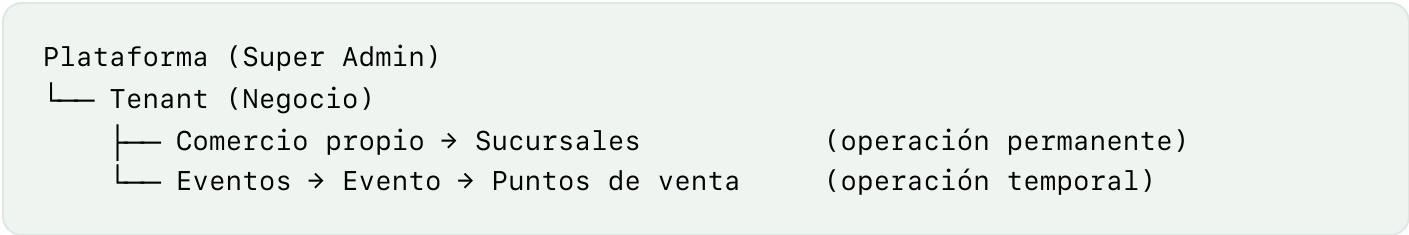
25 de julio de 2026

Fuente: docs/*.md del repositorio

RESUMEN EJECUTIVO

Qué vamos a construir y sobre qué columna vertebral

Un SaaS multi-tenant donde cada negocio (bar, restaurante, discoteca u organizador de eventos) opera aislado con sus propios usuarios, inventario, ventas y reportería, administrado globalmente desde un panel de super admin. Hay dos modalidades: **comercio propio** (operación permanente en sucursales) y **eventos** (contenedores temporales que crean mini bares y restaurantes dentro).



La abstracción que sostiene todo: la sucursal y el punto de venta de un evento son el mismo concepto — la **unidad operativa**. Ventas, inventario, cajas y personal siempre cuelgan de una unidad operativa; la modalidad es un atributo, no una estructura distinta. Así ambas modalidades comparten el 90% del código.

Stack

Monolito Laravel full-stack. Back-office en Livewire (Filament); solo el POS es una PWA

Offline-first

El POS vende sin internet: base local en el navegador, sincronización idempotente por

con JavaScript, por el requisito offline.

lotes, foliado fiscal por bloques por terminal.

República Dominicana

NCF y e-CF (DGII, Ley 32-23), ITBIS 18%, propina legal 10%, exportes 606/607.

Moneda DOP.

DOCUMENTO 01

Visión y alcance

El problema

Bares, restaurantes y discotecas manejan inventario, ventas y personal con herramientas fragmentadas o con papel. En eventos (festivales, conciertos, ferias) el problema se multiplica: barras y cocinas temporales sin control centralizado de inventario ni ventas, con conectividad poco confiable.

Las dos modalidades

MODALIDAD	QUIÉN	CÓMO OPERA
Comercio propio	Restaurante, bar, discoteca, cafetería	Una o varias sucursales; catálogo e inventario por sucursal (con maestro compartido opcional); turnos, cierres diarios, reportería histórica.
Eventos	Organizador de festivales, conciertos, ferias, temporadas	El evento es un contenedor con fechas y presupuesto. Dentro se crean puntos de venta con inventario asignado, personal y terminales propios. Reportería consolidada en vivo; al cerrar: devolución de sobrantes, liquidación y archivo.

Un mismo tenant puede tener ambas modalidades activas (una discoteca que además organiza festivales).

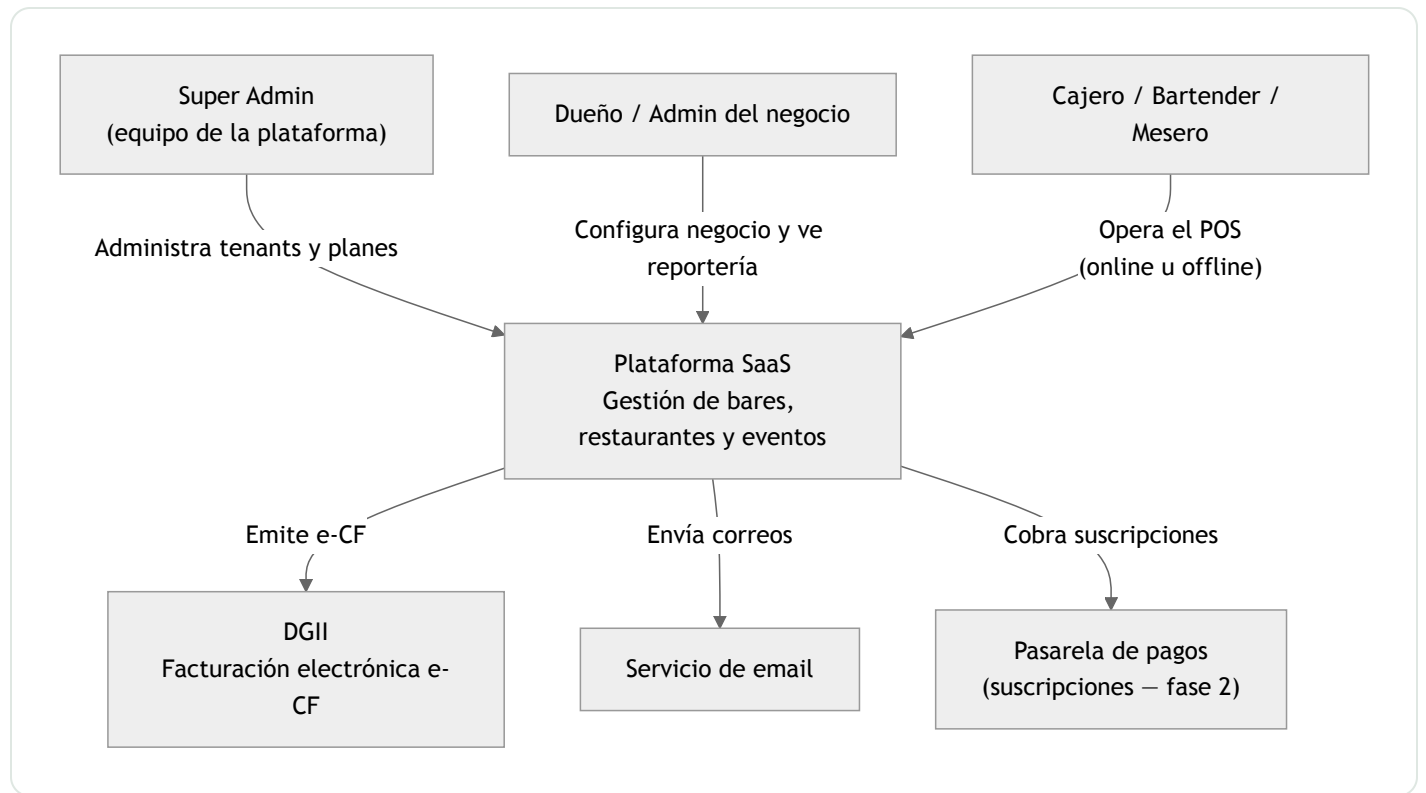
Actores

ACTOR	NIVEL	DESCRIPCIÓN
Super admin	Plataforma	Dueño del SaaS: tenants, planes, facturación del servicio, soporte, métricas globales.
Soporte plataforma	Plataforma	Solo lectura/asistencia a tenants; sin datos sensibles.
Dueño del negocio	Tenant	Control total: sucursales, eventos, usuarios, configuración, reportería completa.
Administrador	Tenant	Gestión operativa delegada según permisos.
Gerente de unidad	Unidad	Administra una sucursal o punto de evento: inventario, personal, cierres.
Cajero / bartender / mesero	Unidad	Opera el POS: órdenes, cobros, su caja.
Almacenista	Tenant / unidad	Compras, recepción, transferencias, conteos.

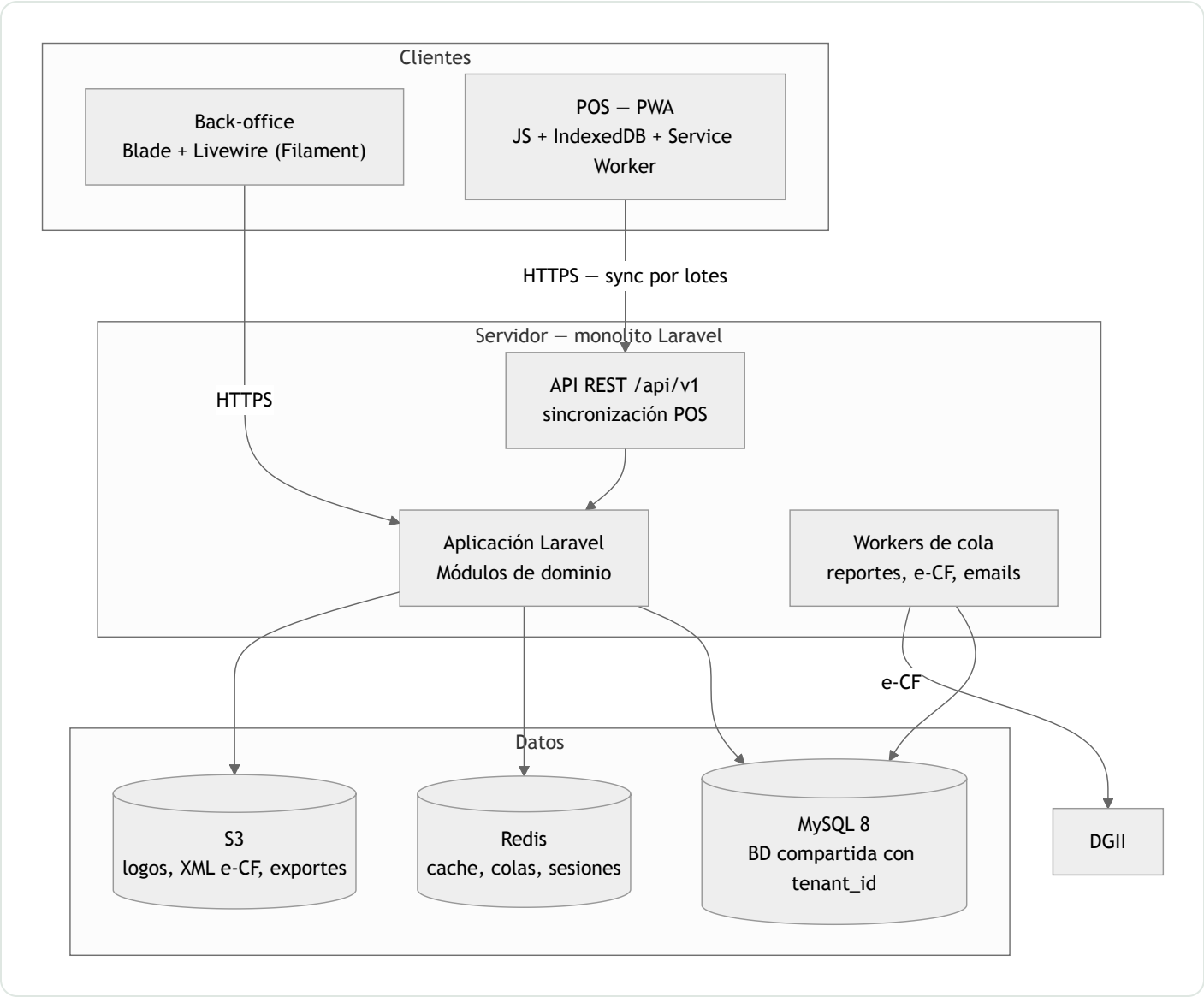
Fases

FASE	CONTENIDO
MVP	Tenants con alta manual · comercio propio con una sucursal · catálogo con recetas e insumos · inventario (compras, ajustes, consumo por venta) · POS offline-first con efectivo/tarjeta y apertura/cierre de caja · NCF · reportería básica (ventas, top productos, cierre Z) · usuarios y roles.
Fase 2	Modalidad eventos completa · multi-sucursal con transferencias · e-CF según calendario DGII · planes self-service con pasarela · app de comandas para meseros.
Fase 3	Pasarelas locales (Azul, CardNET) en el POS · órdenes de compra a proveedores · pronóstico de demanda · API pública.
Fuera de alcance	Delivery y pedidos en línea de clientes finales · nómina · contabilidad formal (se exporta).

Contexto (C4 nivel 1)



Contenedores (C4 nivel 2)

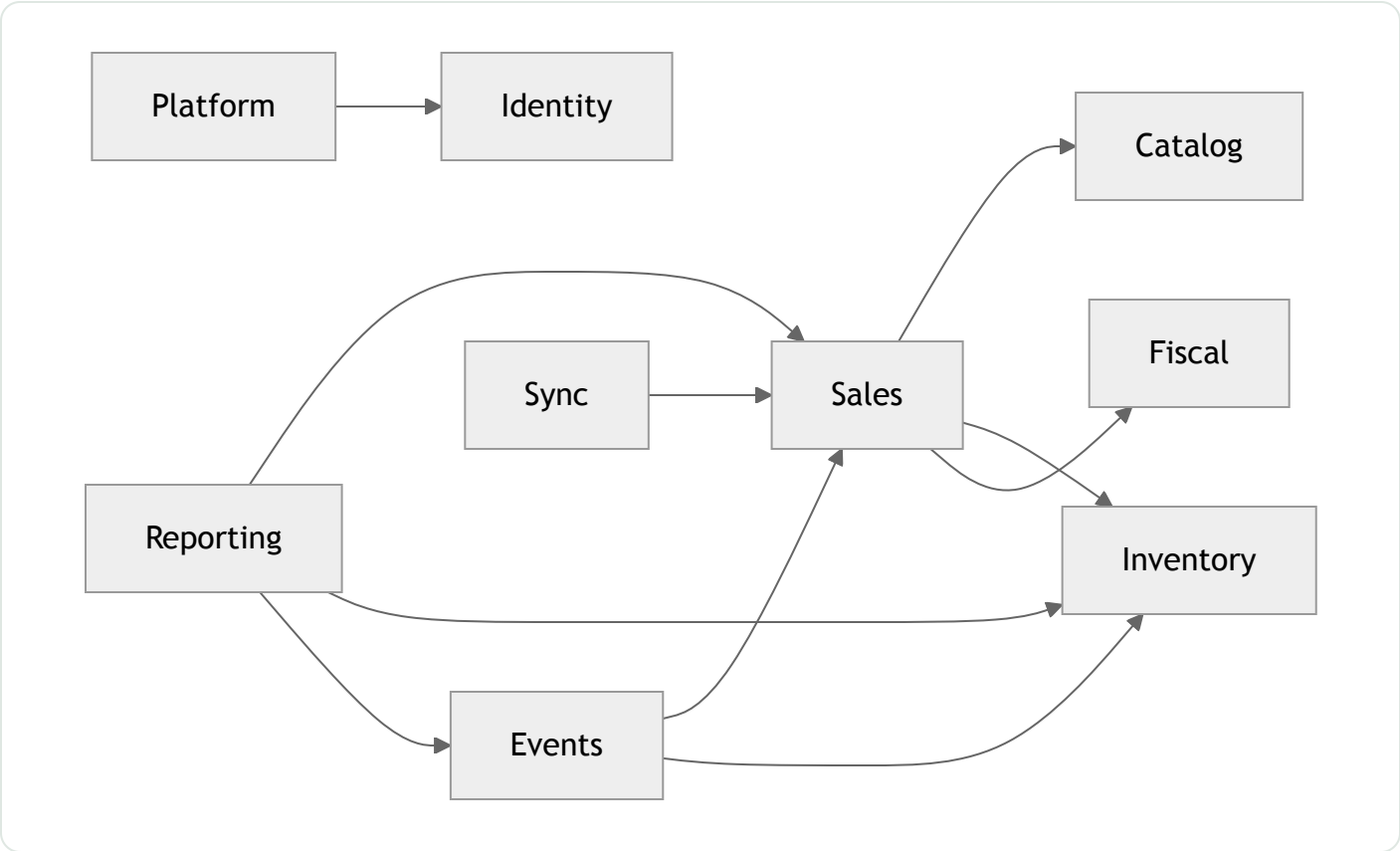


Módulos del monolito

MÓDULO	RESPONSABILIDAD
Platform	Tenants, planes, suscripciones, panel super admin
Identity	Usuarios, roles, permisos, invitaciones (spatie/laravel-permission)
Tenancy	Resolución del tenant, scoping global, contexto de unidad operativa
Catalog	Productos, categorías, precios, recetas (escandallo), modificadores
Inventory	Insumos, stock por unidad, compras, transferencias, mermas, conteos
Sales	Órdenes, pagos, sesiones de caja, mesas/zonas, turnos

Events	Eventos, puntos de venta, asignación de inventario, liquidación
Fiscal	Secuencias NCF, emisión e-CF, exportes 606/607
Reporting	Dashboards, reportes, cierre Z, consolidados de evento
Sync	Endpoint de sincronización del POS, idempotencia

Reglas de dependencia



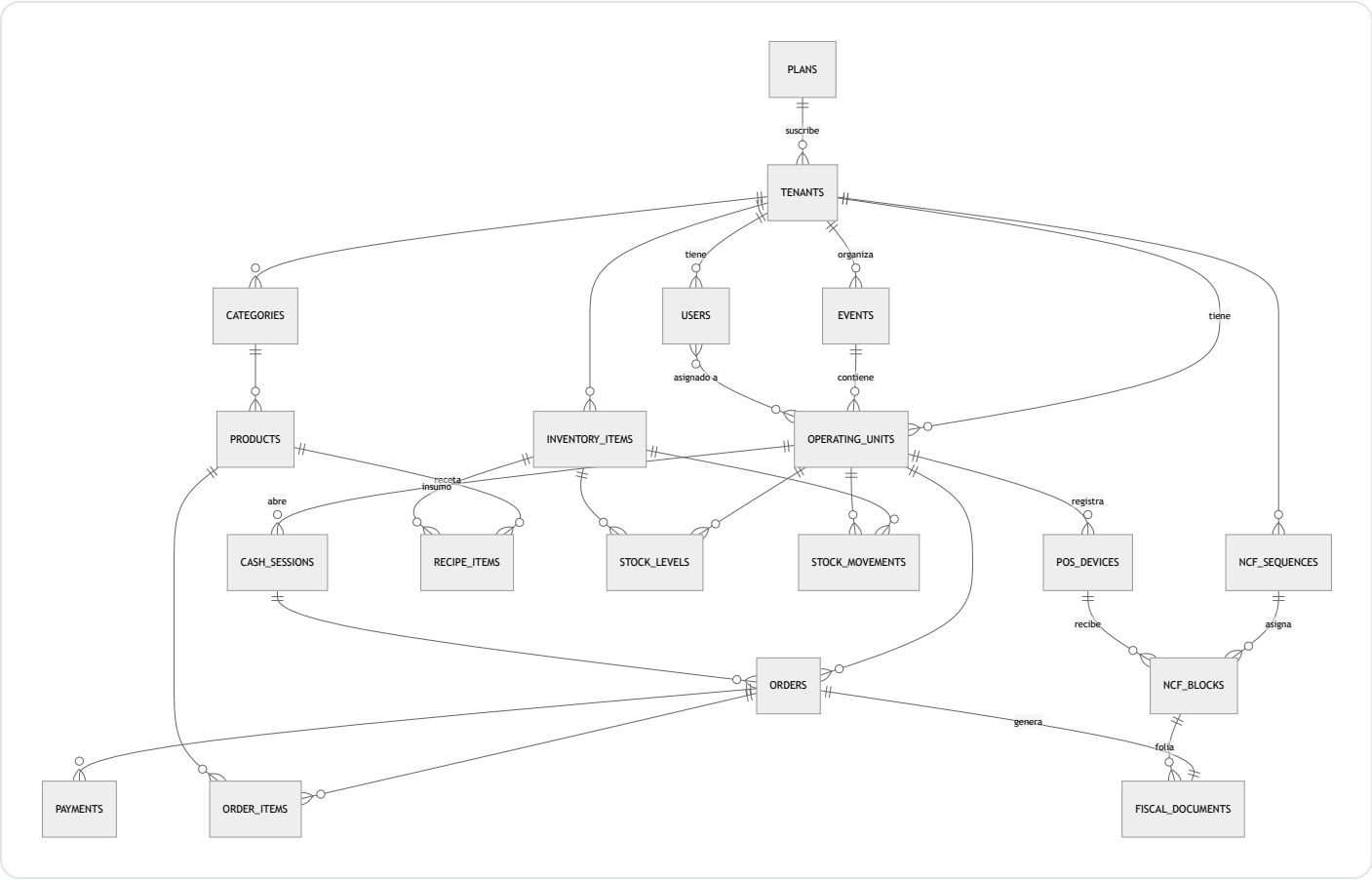
- **Catalog** e **Inventory** no conocen a **Sales** (dependencia en una sola dirección).
- **Fiscal** no conoce ningún dominio de negocio: recibe "documentos a foliar/emitir".
- **Reporting** solo lee — idealmente contra réplicas o tablas agregadas.

DOCUMENTO 03

Modelo de datos

Convenciones: toda tabla de negocio lleva `tenant_id` ; lo transaccional cuelga de `operating_units` ; órdenes y pagos usan **UUID generado en el cliente** (idempotencia

offline); dinero en centavos `BIGINT` , moneda DOP.



Campos clave por entidad

ENTIDAD	CAMPOS QUE DEFINEN EL DISEÑO
tenants	rnc , status (active suspended trial), plan_id
operating_units	event_id nullable (null = sucursal), type (branch event_outlet), status (active closed settled)
events	starts_at , ends_at , status (draft active closed settled)
products	type (simple recipe), price_cents , track_stock
recipe_items	product_id + inventory_item_id + quantity en unidad base del insumo
inventory_items	base_unit (ml g unidad), cost_cents (costo promedio)
stock_movements	type : purchase, sale_consumption, transfer_in/out, waste, adjustment, event_allocation, event_return

orders	UUID, subtotal_cents, itbis_cents, tip_cents, total_cents, sold_at (hora real del POS), synced_at
order_items	unit_price_cents – precio congelado al momento de la venta
payments	UUID, method (cash card transfer other)
cash_sessions	opening_cents, closing_cents, dispositivo y usuario que abre/cierra
pos_devices	device_token, last_sync_at
ncf_sequences / ncf_blocks	tipo (B02, B01, E32...), rango autorizado, vencimiento; bloques por terminal con from/to/used_up_to
fiscal_documents	ncf asignado, status (issued sent accepted rejected voided), xml_path

Decisiones de modelado

1. **La unidad operativa unifica sucursal y punto de evento.** La modalidad es un atributo, no una estructura paralela.
2. **Inventario de eventos = movimientos, no tablas nuevas.** Asignar stock a un punto es `transfer_out` + `event_allocation`; la liquidación se calcula: asignado – vendido (según recetas) – mermas – devuelto = faltante.
3. **Consumo derivado de recetas.** Al pagar, cada línea explota su receta y genera `sale_consumption`. Un producto simple con stock consume 1:1.
4. **Precio congelado en la venta.** Cambiar el catálogo nunca altera ventas históricas.
5. **UUID solo donde nace offline** (órdenes/pagos); el resto usa autoincrementales por rendimiento.
6. **`sold_at` ≠ `created_at`**. La hora fiscal y de reportería es la del POS, no la de llegada al servidor.

DOCUMENTO 04

Roles y permisos

Implementación prevista: `spatie/laravel-permission` con teams (el team es el tenant). Tres niveles de contexto: **plataforma → tenant → unidad operativa**. Un

usuario pertenece a un tenant y puede asignarse a varias unidades; los permisos de unidad solo aplican dentro de las asignadas.

PERMISO	OWNER	ADMIN	EVENT_MGR	UNIT_MGR	WAREHOUSE	CASHIER
Gestionar usuarios del tenant	Sí	Sí	—	—	—	—
Configurar catálogo y precios	Sí	Sí	—	—	—	—
Crear eventos / puntos de venta	Sí	Sí	Sí	—	—	—
Asignar inventario a evento	Sí	Sí	Sí	—	Sí	—
Liquidar / cerrar evento	Sí	Sí	Sí	—	—	—
Compras y recepción	Sí	Sí	—	Sí	Sí	—
Transferencias entre unidades	Sí	Sí	—	Sí	Sí	—
Ajustes y mermas	Sí	Sí	—	Sí	Sí	—
Reportería del tenant completo	Sí	Sí	—	—	—	—
Reportería de su unidad	Sí	Sí	Sí	Sí	—	—
Abrir / cerrar caja propia	Sí	Sí	—	Sí	—	Sí
Vender en POS	Sí	Sí	—	Sí	—	Sí
Anular orden pagada	Sí	Sí	—	Sí	—	—
Descuentos sobre límite	Sí	Sí	—	Sí	—	—
Configurar secuencias NCF	Sí	Sí	—	—	—	—

Registrar dispositivos POS	Sí	Sí	—	Sí	—	—
-------------------------------	----	----	---	----	---	---

Roles adicionales: `super_admin` y `platform_support` a nivel plataforma; `waiter` (comandas sin cobro) llega en fase 2. Roles personalizados por tenant: fase 2.

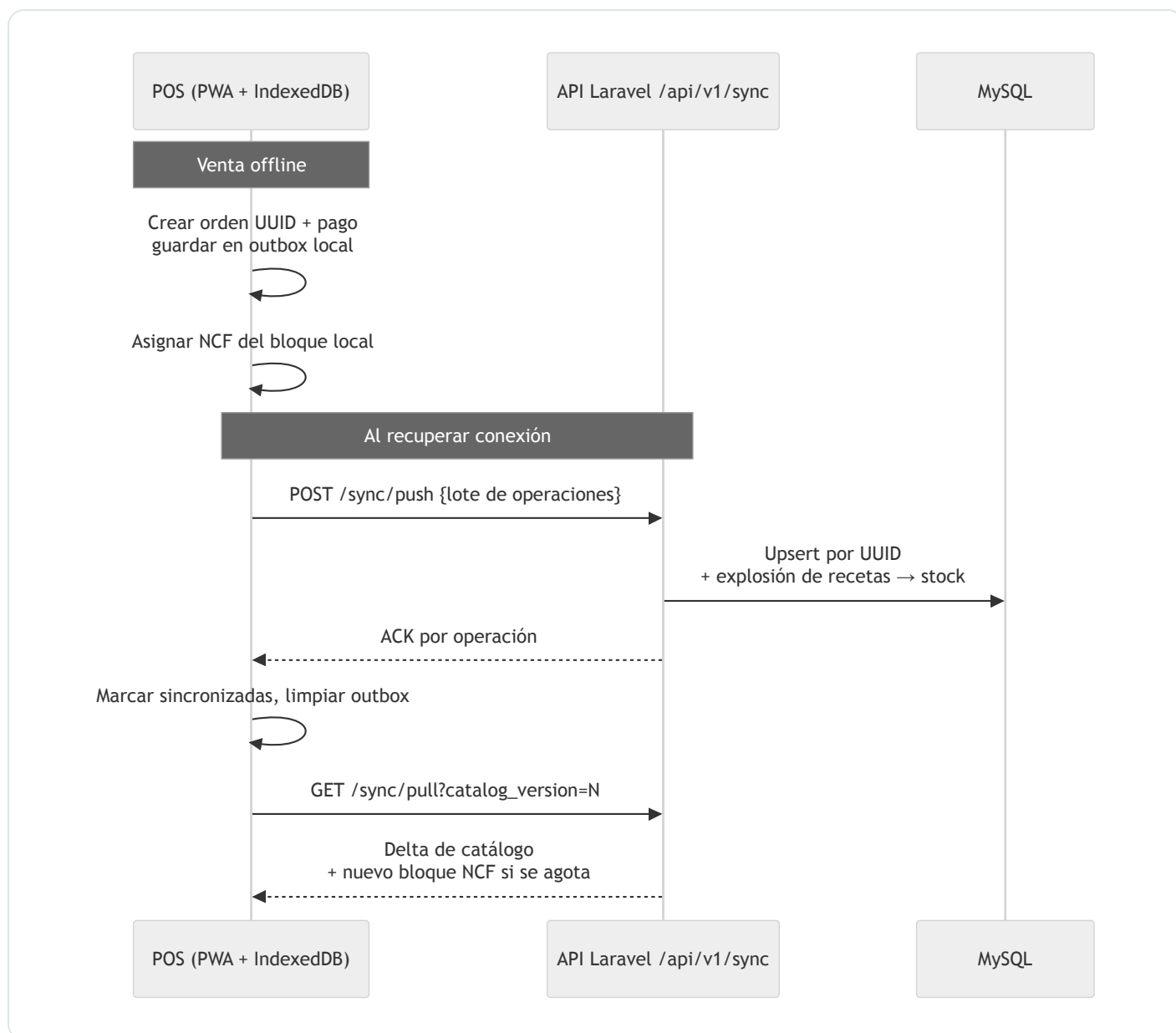
Reglas transversales

- **Scoping absoluto por tenant**, garantizado con global scopes y tests de aislamiento en CI.
- **Acciones sensibles con PIN de supervisor** en el POS: anulaciones, descuentos altos, gaveta sin venta.
- **Auditoría** de toda escritura relevante (spatie/laravel-activitylog).
- **El token del dispositivo no es un usuario**: autentica al aparato; el operador entra con PIN. Ambos quedan registrados en cada venta.

DOCUMENTO 05

POS offline y sincronización

Principios: el POS siempre opera contra su base local (IndexedDB) — la red es una mejora, nunca un requisito para vender. El servidor es la fuente de verdad del catálogo; el POS lo es de sus propias ventas hasta sincronizar. Cada terminal crea órdenes con UUID propio, así que **las ventas no pueden entrar en conflicto**, y la sincronización es idempotente (upsert por UUID: reenviar un lote no duplica nada).

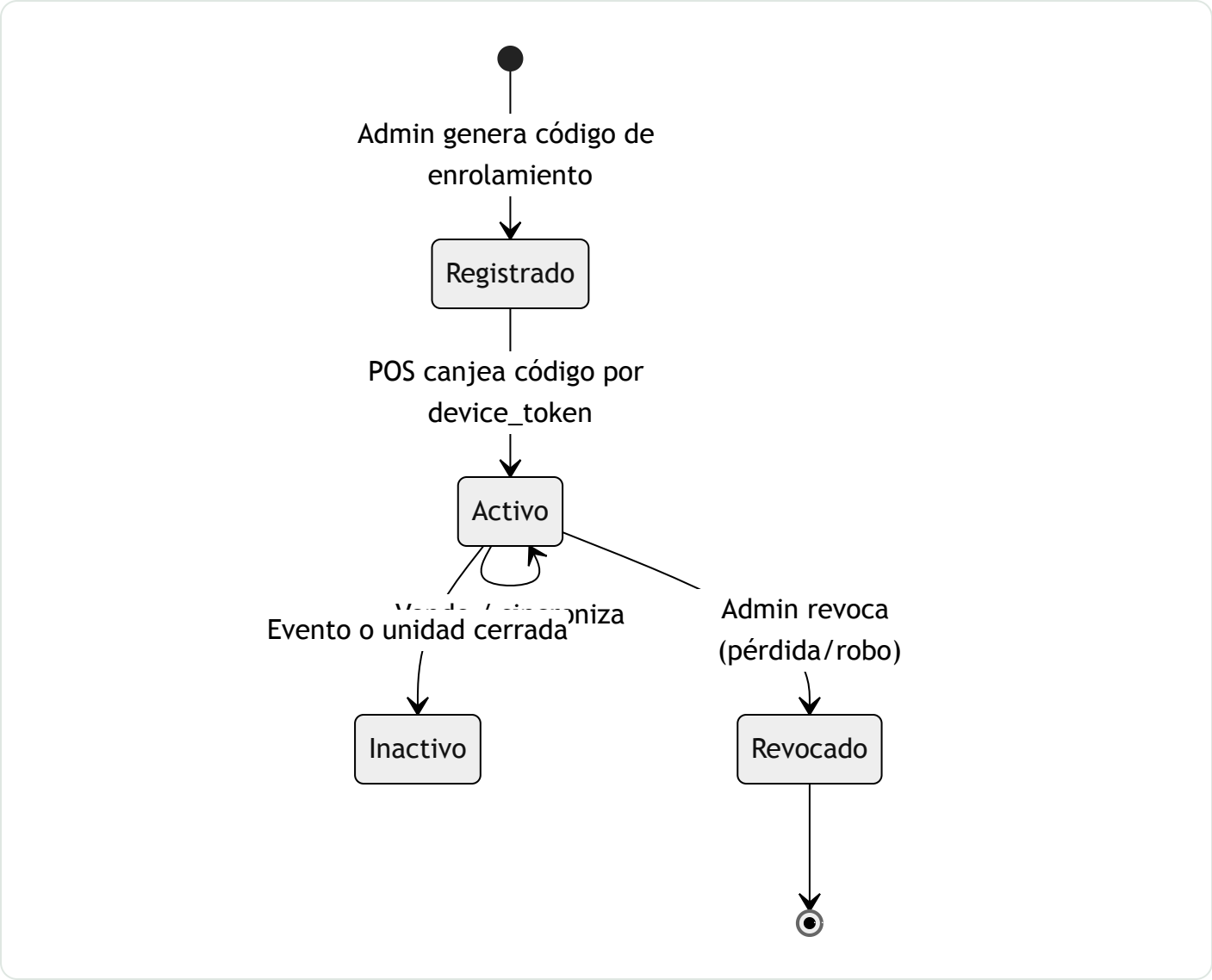


Reglas de resolución

SITUACIÓN	REGLA
Catálogo cambió mientras el POS estaba offline	La venta con el precio del snapshot es válida (precio congelado); el nuevo precio aplica al refrescar.
Producto desactivado mientras offline	Ventas hechas se aceptan; el producto desaparece al refrescar.
Stock negativo al consolidar	Se acepta el movimiento y se alerta descuadre; el conteo físico manda. Nunca se bloquea una venta ya cobrada.
Lote reenviado por timeout	Upsert por UUID: cero duplicados.

Reloj del dispositivo desviado	Se guarda <code>soId_at</code> tal cual + <code>synced_at</code> del servidor; desvíos grandes alertan al gerente.
Dispositivo perdido o robado	Se revoca su token desde el back-office; su bloque NCF sin usar se anula.

Ciclo de vida del dispositivo



LÍMITES EXPLÍCITOS DEL MODO OFFLINE

No hay transferencia de órdenes entre terminales sin red (una mesa abierta vive en un solo terminal hasta sincronizar). La reportería en vivo refleja solo lo sincronizado; el cierre de un evento exige que todos los terminales hayan sincronizado — el sistema lo verifica y lista pendientes. El enrolamiento inicial y la primera sesión requieren conexión.

Fiscalidad – República Dominicana

VALIDACIÓN PENDIENTE

Tasas, montos y calendarios deben validarse con un contador dominicano antes de implementar, y revisarse periódicamente: la normativa DGII cambia.

CONCEPTO	QUÉ ES	IMPACTO EN EL SISTEMA
RNC	Registro Nacional del Contribuyente	Campo del tenant; validación de formato; requerido para B01/E31
NCF	Número de Comprobante Fiscal autorizado por DGII	Secuencias con vencimiento; el sistema folia cada venta
e-CF	Comprobante electrónico (XML firmado, Ley 32-23)	Fase 2: firma digital, envío, acuses; obligatoriedad progresiva
ITBIS	Impuesto general del 18%	Cálculo por línea; exentos configurables; desglose en ticket
Propina legal	10% obligatorio en bares/restaurantes (art. 228 Código de Trabajo)	Línea automática; configurable por unidad; reportería de propinas

Tipos de comprobante

PAPEL (NCF)	ELECTRÓNICO (E-CF)	USO
B02	E32	Consumo – venta típica de POS a consumidor final
B01	E31	Crédito fiscal – cliente con RNC que pide factura
B04	E34	Nota de crédito – anulaciones/devoluciones tras emitir
B14 / B15	E44 / E45	Regímenes especiales / gubernamental

Orden de cálculo del ticket

Subtotal (suma de líneas)
+ Propina legal 10% (sobre subtotal, si la unidad la aplica)

+ ITBIS 18% (sobre subtotal; líneas exentas no suman base)
= Total

Si el precio de carta es "ITBIS incluido" (común en bares), el sistema desglosa hacia atrás — configurable por tenant.

Requisitos del módulo Fiscal

1. Gestión de secuencias NCF: rangos autorizados por tipo, vencimiento, alertas de agotamiento.
2. Foliado offline por bloques por terminal (ver ADR-004).
3. Nota de crédito obligatoria para anular una venta foliada — nunca se borra.
4. Reemisión de consumo (B02) como crédito fiscal (B01) cuando el cliente presenta RNC.
5. Exportes 606/607 por período en formato DGII.
6. e-CF en fase 2: XML firmado con certificado del tenant, envío, acuses y contingencia.
7. La pre-cuenta se marca visiblemente: **"NO VÁLIDO COMO COMPROBANTE FISCAL"**.

DOCUMENTO 07

Diseño de UI y paneles

Tres superficies, tres usuarios, tres contextos — un solo sistema visual (Tailwind + tokens compartidos).

SUPERFICIE	USUARIO	CONTEXTO	PERSONALIDAD
POS (PWA)	Cajero / bartender	De pie, con prisa, poca luz, táctil	Oscuro, botones grandes, cero fricción
Back-office	Dueño / gerente	Sentado, analizando, desktop	Claro, denso en datos, navegable
Super admin	Equipo de la plataforma	Herramienta interna	Funcional, tablas, sin marca de tenant

DECISIÓN TÉCNICA

Filament como base del back-office y el super admin: es Livewire puro (alineado con ADR-001), soporta multi-panel con guards separados y multi-tenancy compatible con nuestro `tenant_id`. Tablas, formularios, filtros y widgets vienen resueltos — acelera el MVP meses. El POS no usa Filament: es la PWA Vue (ADR-003). Filament administra; el POS vende.

POS – decisiones de experiencia

- Tema oscuro por defecto (entornos nocturnos); claro opcional para cafeterías.
- Objetivos táctiles ≥ 56 px, sin hover, sin menús anidados en el flujo de venta.
- Dos modos por unidad: **barra** (grid → cobrar en 2-3 toques) y **restaurante** (mesas/zonas, cuentas abiertas, pre-cuenta).
- Cobro con denominaciones RD\$ (100/200/500/1000/2000), cambio en tipografía gigante, pago dividido; efectivo / tarjeta / transferencia.
- Desglose fiscal siempre visible: subtotal, propina 10%, ITBIS 18%.
- Cambio de operador por PIN sobre el mismo dispositivo; acciones sensibles con PIN de supervisor.
- Estado de sincronización persistente y discreto (verde / ámbar / rojo), nunca modal, nunca bloqueante.
- Teclado numérico propio en pantalla para importes.

Back-office

- Navegación lateral: Dashboard, Ventas, Inventario, Catálogo, Eventos, Reportes, Usuarios, Configuración.
- **Selector de contexto** en la barra superior — cambia entre todo el negocio, una sucursal o un punto de un evento. Es el reflejo directo del concepto de unidad operativa.
- Dashboard por contexto: ventas de hoy, órdenes, ticket promedio, alertas de stock, ventas por hora, top productos.
- Vista de **evento en vivo**: leaderboard de puntos de venta, semáforo de stock, terminales sin sincronizar, botón de liquidación.

Super admin

- Tabla de tenants (estado, plan, unidades, último uso); alta manual en MVP.
- Impersonation ("entrar como") con auditoría.
- Salud de sincronización global: dispositivos con horas sin sync = soporte proactivo.
- Métricas SaaS (MRR, churn, activos) en fase 2 con suscripciones.

Marca blanca ligera

Cada tenant configura logo y color de acento, aplicados a la cabecera del POS, la pantalla de bloqueo, los tickets (con datos fiscales y RNC) y sus correos transaccionales. El back-office mantiene la identidad de la plataforma; white-label completo no es MVP.

DECISIONES DE ARQUITECTURA

ADRs – qué decidimos y por qué

ADR-001 **Monolito Laravel full-stack; PWA solo para el POS** ACEPTADA

El equipo domina Laravel. Todo el backend es un monolito modular; el back-office es Blade + Livewire. El POS es la única excepción: Livewire renderiza en servidor y sin red no hay interfaz, así que offline-first exige una PWA (Vue 3 + IndexedDB + Service Worker) — servida por el mismo Laravel, mismo repo, mismo deploy.

Descartado: todo Livewire con POS online-only (viola el requisito offline) · SPA completa (duplica esfuerzo sin beneficio) · app nativa/Electron (mayor costo; la PWA cubre tablet y desktop) · microservicios (complejidad injustificada).

ADR-002 **Multi-tenancy: base de datos compartida con `tenant_id`** ACEPTADA

Esperamos muchos tenants pequeños/medianos. Una sola BD con `tenant_id` en toda tabla de negocio: global scope automático (trait `BelongsToTenant`), tenant resuelto en middleware, índices compuestos que empiezan por `tenant_id`, y tests de aislamiento obligatorios en CI.

Descartado: BD por tenant (migraciones y backups $\times N$, reportería global compleja, costo alto para tenants pequeños) · esquemas de PostgreSQL (mismo problema y cambia el motor). **Ruta de escape:** como todo lleva `tenant_id`, un tenant gigante puede extraerse a BD dedicada si algún día hace falta.

ADR-003 POS offline-first con outbox idempotente ACEPTADA

Cada operación de venta nace con UUID de cliente y se encola en un outbox local; se envía por lotes cuando hay red y el servidor hace upsert por UUID — reenviar no duplica. Catálogo baja como snapshot versionado (pull por delta). Sin edición concurrente: una orden pertenece a su terminal hasta sincronizar, así que las ventas no generan conflictos. El stock local es indicativo; el real se consolida en el servidor.

Descartado: siempre online (viola el requisito) · CRDTs / motores de sync genéricos (potencia innecesaria: nuestras escrituras son append-only por terminal; el outbox es más simple de auditar en datos fiscales).

ADR-004 Foliado NCF por bloques por terminal; e-CF en fase 2 ACEPTADA

Una secuencia fiscal es un contador único y dos terminales sin red no pueden compartirlo en vivo. Solución: el servidor reserva sub-rangos de la secuencia por terminal; el terminal folia localmente dentro de su bloque, y en cada sync se registra consumo y se reabastece. Folios de terminales revocados se anulan, nunca se reasignan. Costo asumido: los NCF no salen en orden cronológico global — trazabilidad garantizada por terminal + timestamp. El e-CF (fase 2) reutiliza la misma infraestructura, con emisión asíncrona vía cola: una venta jamás espera por DGII.

Descartado: foliar solo en servidor (el POS offline no emitiría comprobante) · foliar al sincronizar (el cliente se va sin comprobante) · una secuencia DGII por terminal (carga administrativa absurda).

Preguntas abiertas

La revisión debería producir respuestas, no solo lectura. Estas son las decisiones que siguen abiertas y quién debería opinar:

- P1** Bloques NCF por terminal, folios anulados y numeración no consecutiva en los reportes 606/607: ¿es aceptable ante DGII tal como está diseñado?

CONTADOR DOMINICANO

- P2** Hardware objetivo del POS: ¿tablets Android, iPads o PCs táctiles? Afecta pruebas de la PWA, teclado e impresión.

EQUIPO + OPERACIÓN

- P3** Impresión de tickets: térmicas de red vs. Bluetooth vs. USB — las PWA tienen límites con USB/Bluetooth; la impresión por red es la vía segura. ¿Confirmamos ese requisito de hardware?

EQUIPO TÉCNICO

- P4** Filament: hacer una prueba de concepto de multi-panel + multi-tenancy con nuestro modelo antes de comprometernos.

EQUIPO TÉCNICO

- P5** ¿La modalidad eventos queda completa en fase 2, o el MVP necesita una versión reducida (un evento, pocos puntos) para validar el diferenciador?

NEGOCIO

- P6** e-CF: ¿integración directa con DGII o vía proveedor de facturación autorizado? (ADR pendiente; decidir cuando el calendario alcance a nuestros tenants.)

NEGOCIO + CONTADOR

- P7** ¿Precios de carta con ITBIS incluido por defecto? Común en bares; el sistema soporta ambos, falta decidir el default.

NEGOCIO

- P8** Nombre del producto: "ProyectoRest" es provisional.

ProyectoRest · Documento de revisión v0.1 · 25 de julio de 2026 · Generado a partir de `docs/01...07` y `docs/adr/ADR-001...004` del repositorio. Los archivos Markdown son la fuente de verdad; este HTML es la vista consolidada para revisión.